

PROGRAMACIÓN I

Proyecto Integrador

Lineamientos generales

ETAPA 1 · PROTOTIPO FUNCIONAL EN MEMORIA

Ing. Maria Eugenia Varando

Segundo cuatrimestre 2026

1. Sentido de la primera iteración

La primera iteración tiene como finalidad convertir una situación problemática en un prototipo funcional desarrollado en Python. El equipo deberá demostrar que puede analizar el problema, representar sus datos mediante estructuras adecuadas y organizar la solución en funciones y módulos.

Criterio central: La entrega debe resolver un problema concreto. No alcanza con mostrar opciones de menú sin procesamiento real ni con presentar funciones aisladas que no formen un sistema integrado.

2. Contenidos que deben evidenciarse (obligatorios)

- **Funciones y parámetros.** Descomposición del problema en operaciones pequeñas y cohesivas.
- **Módulos.** Separación entre datos, lógica del problema e interacción con el usuario.
- **Listas y matrices.** Representación de colecciones y datos organizados en filas y columnas.
- **Cadenas de caracteres.** Normalización, validación y presentación de información.
- **Tuplas.** Representación de datos que no deben modificarse durante la ejecución.
- **Comprensión, slicing y lambda.** Uso justificado en al menos una operación del proyecto.
- **Ciclos y decisiones.** Menú repetitivo, búsquedas, recorridos, acumulaciones y selección.
- **Git y GitHub.** Registro progresivo del desarrollo y participación verificable de los integrantes. (manejo de un repositorio para presentar el proyecto en etapa 1 y 2)

3. Alcance funcional mínimo

Cada equipo deberá definir un sistema con ingreso de datos por consola o random que incluya, como mínimo, las siguientes capacidades:

1. Presentar información general del dominio y los elementos que administra el sistema.
2. Registrar datos ingresados por el usuario (actualización).
3. Consultar y mostrar los datos cargados con un formato claro (listados).
4. Realizar al menos tres cálculos o indicadores relevantes para el problema (porcentajes, promedios, máximos, etc).
5. Buscar un elemento por un identificador o criterio definido.
6. Detectar una condición destacable: máximo, mínimo, límite superado, estado crítico u otra regla del dominio.
7. Generar un resumen general del procesamiento (informes finales al menos 5).
8. Mantener un menú activo hasta que el usuario seleccione la opción de salida.

4. Representación de datos esperada

La información deberá mantenerse únicamente en memoria durante la ejecución. La selección de estructuras deberá justificarse en la presentación.

Estructura	Uso esperado
Lista	Colección de elementos, resultados o registros simples. Manejar listas HOMOGÉNEAS
Matriz	Datos relacionados por dos dimensiones: elementos y períodos, categorías y mediciones, productos y sucursales, etc. Manejar matrices HOMOGÉNEAS
Tupla	Datos fijos o de configuración que no deben modificarse.
Cadena	Códigos, nombres, categorías, descripciones y datos formateados.

Límite de la etapa: No se exige persistencia. Al cerrar el programa, los datos pueden perderse. Archivos,

diccionarios y conjuntos se incorporarán en la segunda iteración.

5. Requisitos de diseño modular

La solución no deberá desarrollarse íntegramente dentro de main. Como mínimo, se espera la siguiente separación:

Estructura mínima sugerida

```
proyecto_etapa1/
|
├── main.py          # menú, entradas y coordinación general
├── datos.py         # constantes y datos iniciales
├── operaciones.py   # validaciones, búsquedas, cálculos e informes
└── README.md        # descripción, integrantes e instrucciones
```

- Cada función debe cumplir una responsabilidad concreta.
- Los nombres de funciones y variables deben expresar su propósito.
- Las funciones de cálculo no deberían depender innecesariamente de input o print.
- Los mensajes para el usuario deben concentrarse principalmente en main o en funciones de presentación.
- No se debe repetir código cuando una función puede reutilizarse.

6. Validaciones obligatorias

- Opciones de menú inexistentes.
- Identificadores o códigos no registrados.
- Valores numéricos con formato incorrecto.
- Valores fuera del rango permitido.
- Campos vacíos cuando el dato sea obligatorio.
- Categorías, estados o respuestas que no pertenecen al conjunto admitido.
- Inconsistencias internas en la estructura de datos.

Los errores de carga no deberán finalizar abruptamente el programa. El usuario deberá recibir un mensaje comprensible y poder continuar operando.

7. Procesamientos mínimos

Tipo de procesamiento	Cantidad mínima	Ejemplo genérico
Acumulación	2	Total general y total por categoría.
Conteo	1	Cantidad de elementos que cumplen una condición.
Búsqueda	1	Localizar un elemento por código.
Máximo o mínimo	1	Elemento con mayor valor registrado.
Detección de condición	1	Valores que superan un límite.
Ordenamiento o ranking	1	Listado de mayor a menor mediante lambda u otro criterio.

8. Trabajo con GitHub

- Crear un repositorio compartido antes de iniciar la implementación.
- Incluir un README con el problema, alcance, integrantes y forma de ejecución.

- Realizar commits pequeños y descriptivos durante todo el proceso.
- Evitar que una sola persona concentre la totalidad de los cambios.
- Resolver los conflictos de integración con participación del grupo.

9. Entregables

Entregable	Contenido requerido
Repositorio	Código ejecutable, módulos, README y commits progresivos.
Documento de análisis	Problema, objetivo, usuarios, datos, reglas y alcance.
Casos de prueba	Entradas válidas, inválidas y casos límite con resultado esperado.
Evidencia individual	Funciones desarrolladas, decisiones, dificultades, commits y reflexión.
Presentación grupal	Demostración del sistema y explicación de estructuras, validaciones y distribución del trabajo. (FECHA PRIMER PARCIAL)

10. Presentación de la iteración

Duración sugerida: entre 8 y 10 minutos. Todos los integrantes deberán participar y estar en condiciones de responder preguntas sobre el funcionamiento completo del proyecto.

1. Presentación breve del problema y de los usuarios del sistema.
2. Explicación de la representación de los datos.
3. Demostración de una carga válida y una carga inválida.
4. Ejecución de los principales cálculos e informes.
5. Explicación de la modularización y de una función relevante.
6. Descripción de la participación de cada integrante.
7. Identificación de las mejoras previstas para la segunda iteración.

Se aprueba con la validación individual del resultado. TODOS LOS INTEGRANTES TIENEN QUE CONOCER LA TOTALIDAD DEL PROYECTO.

11. Rúbrica

Criterio	Ponderación
Análisis del problema y alcance	10 %
Funciones y modularización	20 %
Uso de listas, matrices y tuplas	20 %
Validaciones y excepciones	15 %
Procesamientos e informes	20 %
GitHub y trabajo colaborativo	5 %
Documentación y presentación	10 %

12. Lista de control antes de entregar

Control	Verificación
---------	--------------

Control	Verificación
☐	El proyecto se ejecuta desde main.py sin errores.
☐	El menú permite acceder a todas las funcionalidades.
☐	La solución está dividida en funciones y módulos.
☐	Se utilizan listas, una matriz y tuplas de manera justificada.
☐	Las entradas incorrectas se controlan sin cerrar el programa.
☐	Existen cálculos, búsquedas, acumulaciones y una detección de condición.
☐	El repositorio contiene commits de todos los integrantes.
☐	El README explica cómo ejecutar y probar el sistema.
☐	Los casos de prueba incluyen datos válidos, inválidos y límites.
☐	Todos los integrantes pueden explicar el funcionamiento general.(CONDICION EXCLUYENTE PARA APROBAR)

La primera iteración debe demostrar una solución integrada, comprensible y funcional, aun cuando los datos todavía no se conserven entre ejecuciones.